



**UNIVERSIDAD TECNOLÓGICA DE PANAMÁ**

**FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES**

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

**Asignatura:**
Sistemas Operativos

**Asignación:**
Laboratorio #3A: Planificación Real en Linux

**Profesor:**
Prof. Isacio Tamayo

**Estudiante:**
James Tack

**Grupo:**
1IL143

**Fecha de entrega:**
18 de septiembre de 2026

2do Semestre 2026

## De qué se trata el laboratorio

El caso de la guía pone a un sysadmin junior frente a un servidor de NovaTech que responde lento porque un servicio pesado se come la CPU. `sha256sum` leyendo de `/dev/urandom` hace de ese servicio. La tarea es entender cómo decide Linux quién usa el procesador y manipular esa decisión en caliente con `nice`, `renice`, `chrt` y `taskset`, sin reiniciar nada.

Trabajé en la misma VM del Lab 1 (`vm-sistemas-operativos`, Ubuntu 18.04 con kernel 4.15.0-212-generic y 2 CPU), entrando por `vagrant ssh`. Para cada paso seguí el ciclo que pide la guía: anotar qué esperaba ver, ejecutar, comparar y explicar la diferencia cuando la hubo. Hubo varias, y están documentadas en su fase.

### Preparación de la VM

Al levantar la máquina noté que respondía muy lento. El promedio de carga estaba por encima de 8 sin que yo hubiera lanzado nada, y `ps` mostró la causa. El contenedor `tarea2-app` de la Tarea 2 seguía corriendo dentro de la VM con una veintena de procesos `stress.py` a nice 19. Como iba a medir prioridades y reparto de CPU, lo detuve y le quité el reinicio automático para que no volviera a arrancar con la máquina:

```
sudo docker stop tarea2-app
sudo docker update --restart=no tarea2-app
```

Con eso la carga bajó a 0,56. Después instalé lo que pide la guía. `util-linux` ya estaba en su última versión, y dejé impresas las versiones de las tres herramientas.

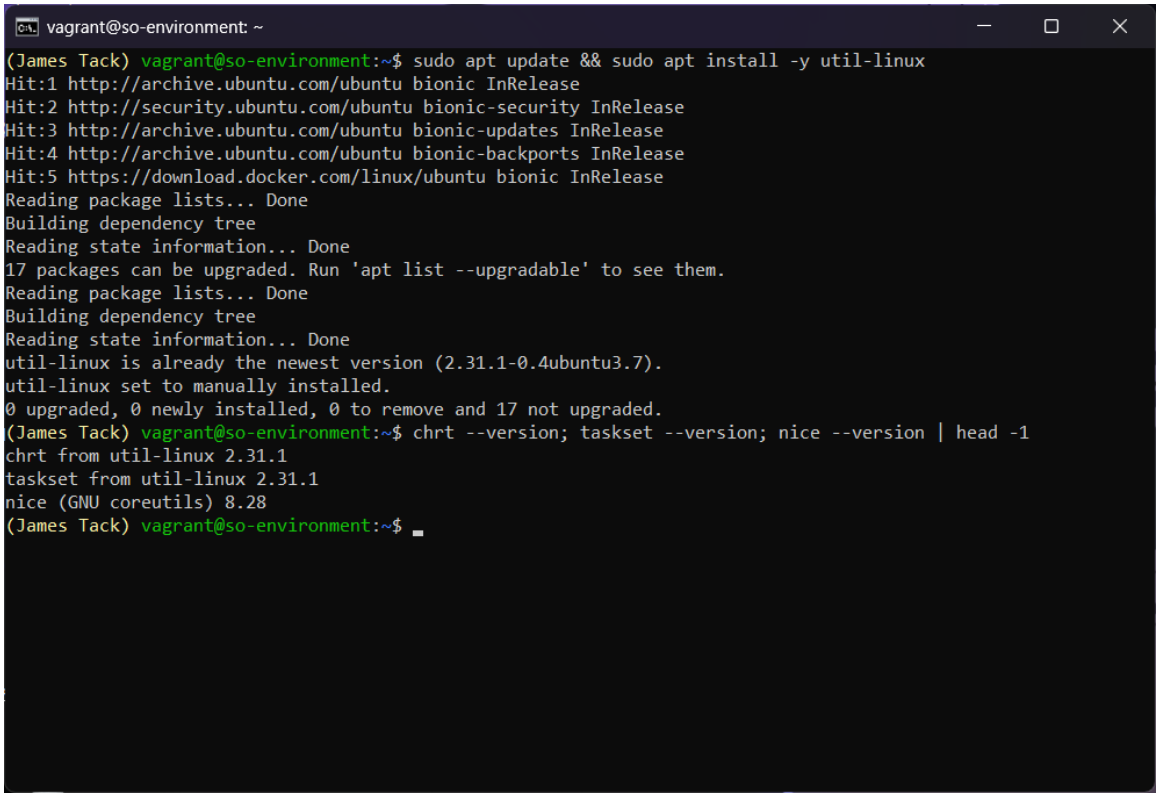



*Figura 1. util-linux 2.31.1 ya instalado: chrt y taskset vienen de ahí, nice de coreutils 8.28.*

## Fase 1. Diagnosticar quién usa la CPU

### 1.1 y 1.2 Proceso pesado y su prioridad

Lancé el proceso que simula el servicio de recomendaciones en segundo plano y consulté su prioridad.

Anticipé `NI` 0 y `PRI` 20, que es lo que dice la tabla de autocomprobación de la guía.

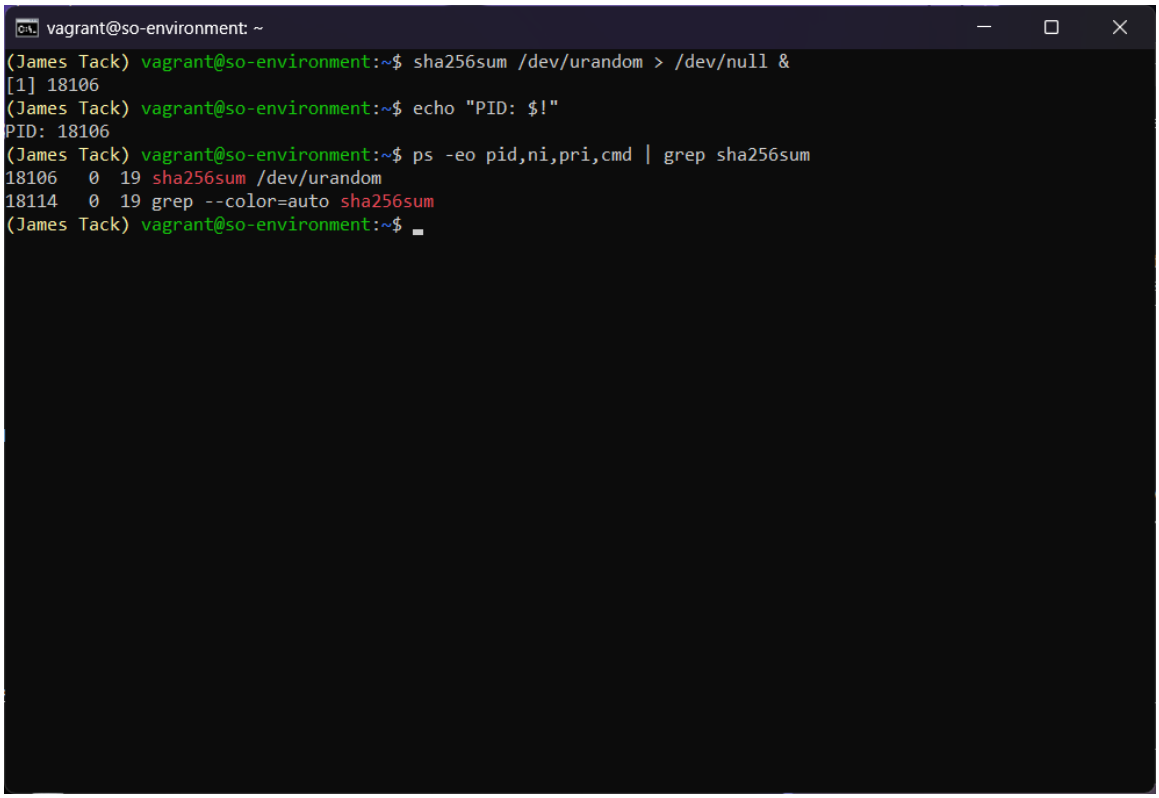



*Figura 2. El proceso 18106 arranca con NI 0; ps muestra PRI 19.*

El `NI` salió en 0 como esperaba, pero `PRI` salió en 19 y no en 20. La diferencia no viene del kernel sino de `ps`, porque la columna `pri` de procps invierte la escala y muestra 139 menos la prioridad interna del kernel, de modo que en esa columna un número más alto significa más prioridad. La columna `priority` sí muestra el valor que usa el kernel (prioridad interna menos 100), y ahí el proceso aparece con 20, igual que en `htop`. Lo comprobé en la fase siguiente.

#### Entregable 1. Niceness por defecto

Un proceso nuevo hereda el niceness de su padre, y como la shell de un usuario corre con 0, el valor por defecto es 0. Significa que nadie pidió nada. El proceso no se ofrece a ceder CPU ni reclama más de la cuenta. Para el planificador CFS, nice 0 equivale a un peso de 1024, que es la referencia contra la que se comparan los demás pesos. En la escala interna del kernel ese proceso queda con prioridad estática 120, la mitad del rango que va de 100 (nice -20) a 139 (nice 19).

## Fase 2. Degradar el proceso problemático con nice

### 2.1 y 2.2 Proceso con nice 10

Agregué la segunda consulta con la columna `priority` para aclarar lo de la fase anterior. El `[s]` en el patrón de `grep` evita que el propio `grep` aparezca en la lista.

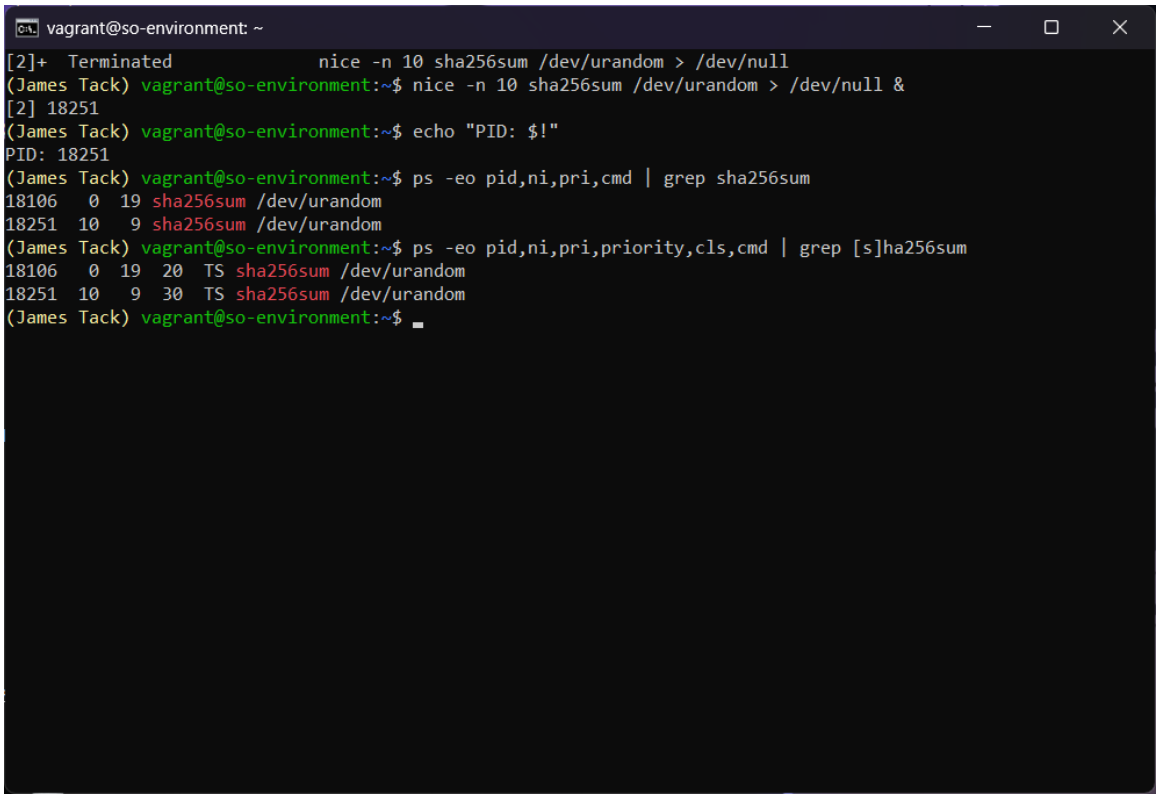



*Figura 3. El proceso 18251, lanzado con nice 10, frente al 18106 con nice 0.*

**Entregable 2. Comparación de prioridades**

1. El proceso lanzado con `nice -n 10` tiene niceness 10.

2. En la columna `pri` de `ps` tiene 9 contra 19 del proceso normal. En la columna `priority`, que es la escala del kernel, tiene 30 contra 20. La guía anticipa un `PRI` mayor para el proceso amable, y eso se cumple en `priority` y en `htop`; en `pri` pasa lo contrario porque la escala va al revés. En el fondo las dos columnas dicen lo mismo, que el proceso con nice 10 queda 10 puestos por debajo.

3. En la práctica, nice 10 significa que el proceso pesa mucho menos cuando compite. CFS reparte el tiempo de CPU en proporción al peso de cada tarea, y el peso de nice 10 es 110 frente a 1024 de nice 0. Si los dos procesos se disputan el mismo núcleo, el de nice 10 se queda con cerca de 110/1134, un 10 % del tiempo, y el normal con el 90 %. La palabra importante es "compiten". En esta VM hay dos núcleos y en este momento solo dos procesos pesados, así que cada uno tuvo un núcleo para sí y los dos corrieron cerca del 100 %. El niceness no frena a nadie cuando sobra CPU; solo decide el reparto cuando falta.

## 2.3 Intentar subir la prioridad con renice

Probé primero lo que pide la guía y después un par de variantes para entender dónde está exactamente la frontera:

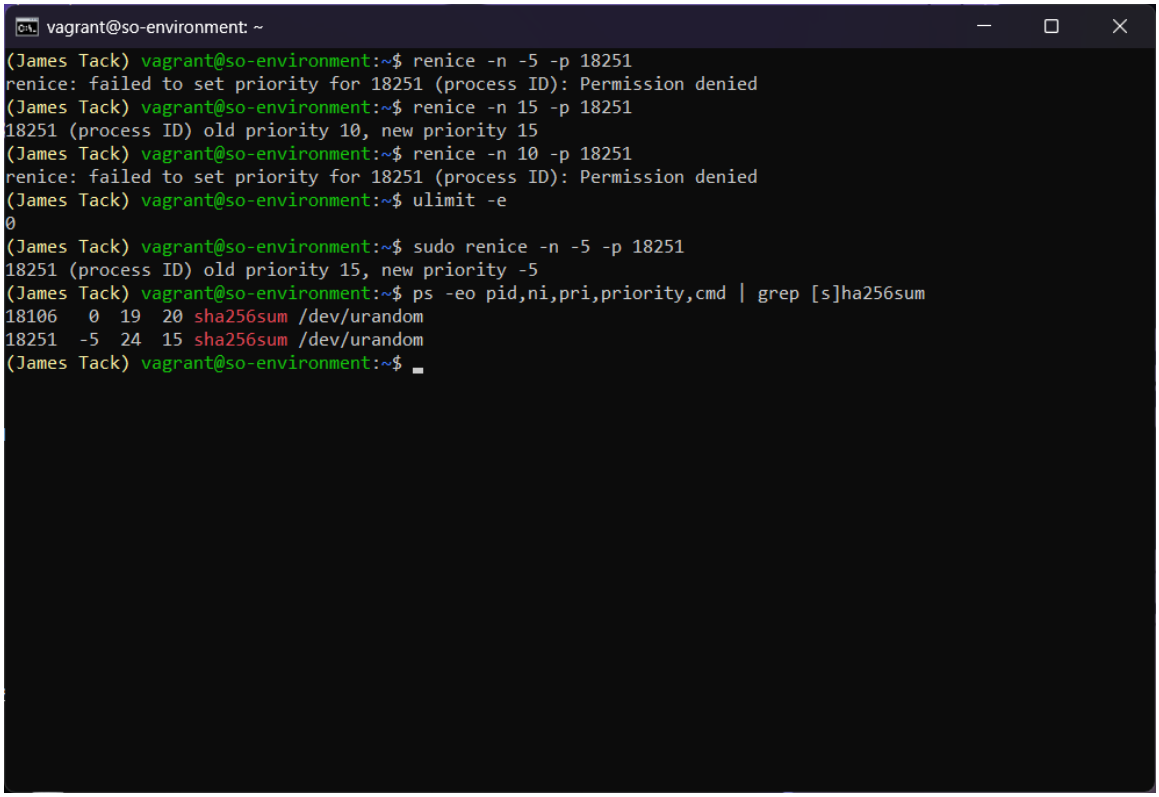



*Figura 4. renice sin privilegios: bajar prioridad funciona, subirla no, ni siquiera de vuelta al valor anterior.*

El resultado de `renice -n -5` fue el esperado, `Permission denied`. Lo que no esperaba es que tampoco me dejara volver de 15 a 10 después de haber subido el nice yo mismo. Una vez que un usuario cede prioridad, no la recupera. `ulimit -e` explica por qué. Es el límite `RLIMIT_NICE` del proceso y vale 0. El kernel solo deja bajar el nice a quien tenga la capacidad `CAP_SYS_NICE` o un `RLIMIT_NICE` suficiente, y con 0 el usuario normal no puede bajarlo ni un punto, sin importar el valor de partida. Con `sudo` el cambio pasó sin problemas y el proceso quedó con NI -5, `priority` 15 y `pri` 24.

### Entregable 3. Análisis del error de permisos

Subir el nice es renunciar a CPU, y renunciar a algo propio no perjudica a nadie, por eso cualquier usuario puede hacerlo. Bajarlo es quitarle CPU a los procesos de los demás usuarios, incluidos los del sistema, y eso requiere autoridad sobre toda la máquina. Si cualquiera pudiera ponerse nice -20, el primer usuario que lo hiciera con un proceso de cómputo intensivo se llevaría casi todo el procesador, porque con peso 88761 frente a 1024, un proceso a nice -20 recibe unas 87 veces más tiempo que uno normal en el mismo núcleo. En un servidor compartido eso deja sin respuesta la shell del administrador, el servidor web y los demonios del sistema.

La regla de que ni siquiera se puede recuperar la prioridad cedida tiene la misma lógica. Si un usuario pudiera volver libremente a su valor anterior, el límite sería fácil de burlar lanzando procesos con nice alto y bajándolos después.

La solución es la elevación de privilegios que se vio en el Lab 2, `sudo`, que ejecuta `renice` como root, y root tiene `CAP_SYS_NICE`:

```
sudo renice -n -5 -p <PID>
```

La alternativa más fina es darle a un usuario o grupo concreto un margen en `/etc/security/limits.conf` (la línea `nice`), que sube su `RLIMIT_NICE` sin darle root completo.

## Fase 3. Diagnosticar la política con chrt

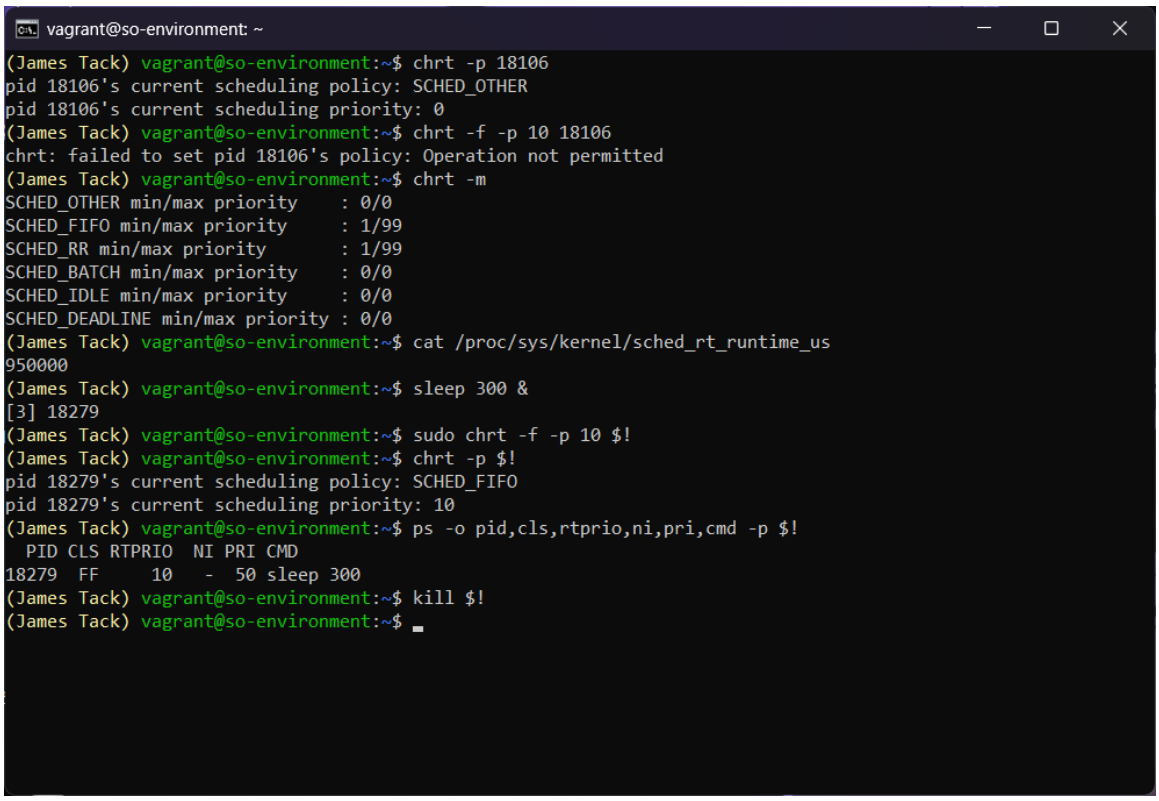



*Figura 5. chrt: SCHED_OTHER por defecto, FIFO denegado sin privilegios y aceptado con sudo sobre un proceso inofensivo.*

La consulta devolvió lo esperado, `SCHED_OTHER` con prioridad 0, y el cambio a FIFO falló con `Operation not permitted`. Según `chrt -m`, `SCHED_FIFO` y `SCHED_RR` admiten prioridades de 1 a 99, mientras que `SCHED_OTHER`, `SCHED_BATCH` y `SCHED_IDLE` solo admiten 0, porque en ellas la prioridad se controla con el nice y no con este número.

Para confirmar que el fallo era solo de permisos, repetí el cambio con `sudo`, pero sobre un `sleep 300` y no sobre `sha256sum`. `sleep` no usa CPU, así que ponerlo en FIFO no tiene consecuencias. Hacerlo con el proceso de cómputo intensivo, en una VM que ya se había puesto lenta con carga

normal, habría sido buscarme el problema de la pregunta siguiente. El `sleep` quedó como `SCHED_FIFO` con prioridad 10, `ps` lo lista con clase `FF`, `rtprio` 10 y `pri` 50, por encima de cualquier proceso normal.

**Entregable 4. SCHED_FIFO en un loop infinito**

Un proceso `SCHED_FIFO` corre hasta que se bloquea, termina, cede la CPU por voluntad propia o llega otro de tiempo real con más prioridad. No tiene quantum. Todos los procesos `SCHED_OTHER`, sin importar su nice, esperan a que las tareas de tiempo real del núcleo terminen, así que un FIFO en loop infinito se queda con su núcleo de forma indefinida. En una máquina de un solo núcleo eso congela el sistema completo: no responde la shell, no entra una sesión SSH para matar el proceso y los hilos del kernel que corren con prioridad normal tampoco avanzan. En una de varios núcleos el daño se limita al núcleo tomado, pero los hilos del kernel atados a ese núcleo se quedan sin turno y el kernel empieza a reportar bloqueos.

Lo único que evita el congelamiento total es el límite de tiempo real que se ve en la captura. `sched_rt_runtime_us` vale 950000 sobre un período de 1000000, lo que significa que las tareas de tiempo real pueden usar como mucho 0,95 s de cada segundo. El 5 % restante queda para los procesos normales. Así, el administrador puede, aunque muy lento, entrar y hacer `kill`. Si alguien pone ese valor en -1 para desactivar el límite, desaparece la red de seguridad.

## Fase 4. Afinidad de CPU con taskset

### 4.1 y 4.2 Anclar un proceso al núcleo 0

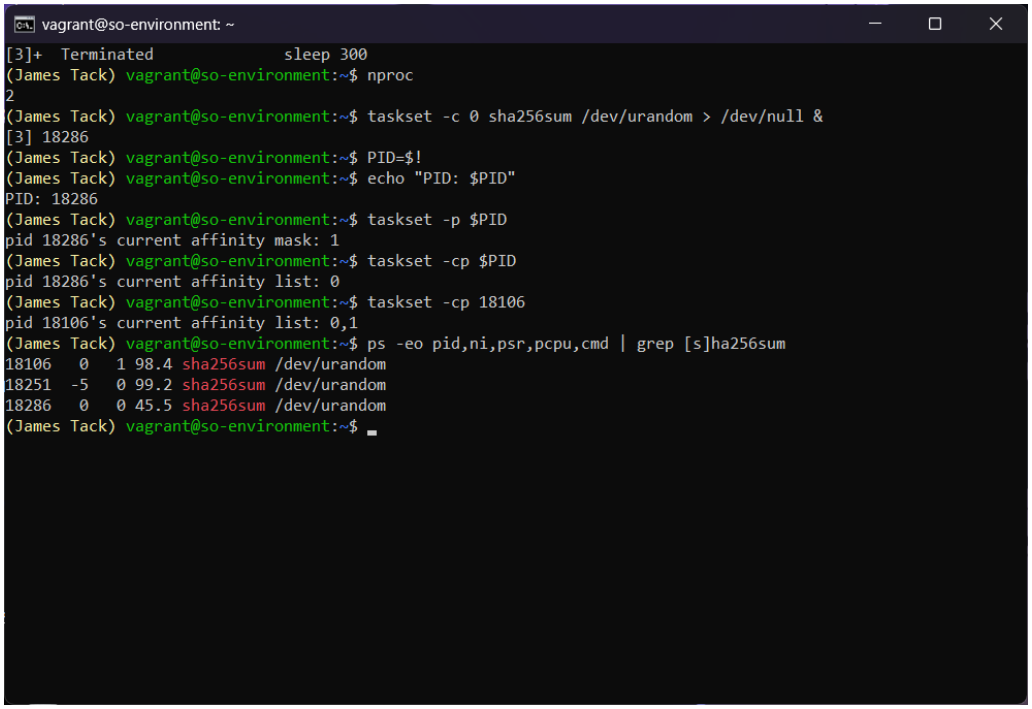



*Figura 6. El proceso 18286 queda anclado al núcleo 0; el 18106 puede usar los dos.*

`taskset -p` devuelve la máscara en hexadecimal (1, es decir, solo el bit del núcleo 0) y `taskset -cp` la traduce a lista (0). La guía anticipa algo como `0-0`; mi versión de `taskset` imprime directamente `0`, que es lo mismo. Como comparación consulté el proceso de la fase 1, que no fue anclado y tiene la lista `0,1`. La columna `psr` de `ps` indica en qué núcleo corrió cada proceso por última vez.

### 4.3 Observar con htop

Activé la columna del procesador en `htop` (en el menú F2 aparece como `PROCESSOR`) y lo abrí con los tres procesos pesados corriendo. El primer intento no sirvió como evidencia.

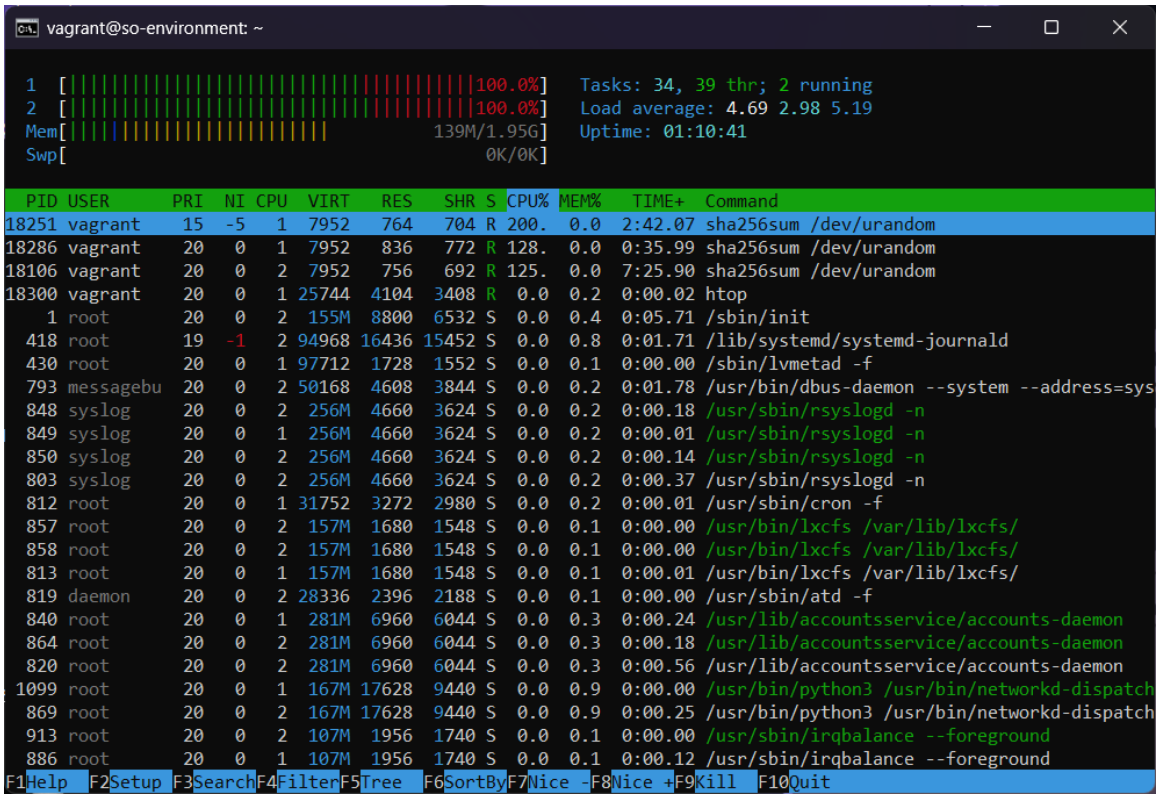



*Figura 7. htop con tres procesos pesados en dos núcleos: la pantalla se congeló después del primer cuadro.*

Los dos núcleos quedaron al 100 % y la pantalla no volvió a refrescarse. El reloj de `Uptime` se quedó detenido más de un minuto y los porcentajes de 200 % son el artefacto del primer cuadro, antes de que `htop` tenga dos muestras para comparar. En ese mismo momento una conexión SSH nueva tardaba más de tres minutos en abrir. Revisé el log de VirtualBox y encontré la explicación del lado del anfitrión: `HM: HMR3Init: Attempting fall back to NEM: AMD-V is not available`. Como en Windows está activo Hyper-V (lo usan WSL2 y Docker Desktop), VirtualBox no tiene acceso directo a la virtualización del procesador y corre la VM a través de la API de Hyper-V. Con los dos procesadores virtuales ocupados al 100 %, esa capa entrega el tiempo a la VM a trompicones y todo lo interactivo se congela. Sin embargo, dentro de la VM el planificador hacía su trabajo. Lo que no llegaba era CPU real desde afuera.

Para poder observar la afinidad terminé los dos procesos que no estaban anclados y lancé un segundo proceso también anclado al núcleo 0. Así la carga quedó concentrada en un solo núcleo, que es justo el escenario de la tercera pregunta de esta fase.

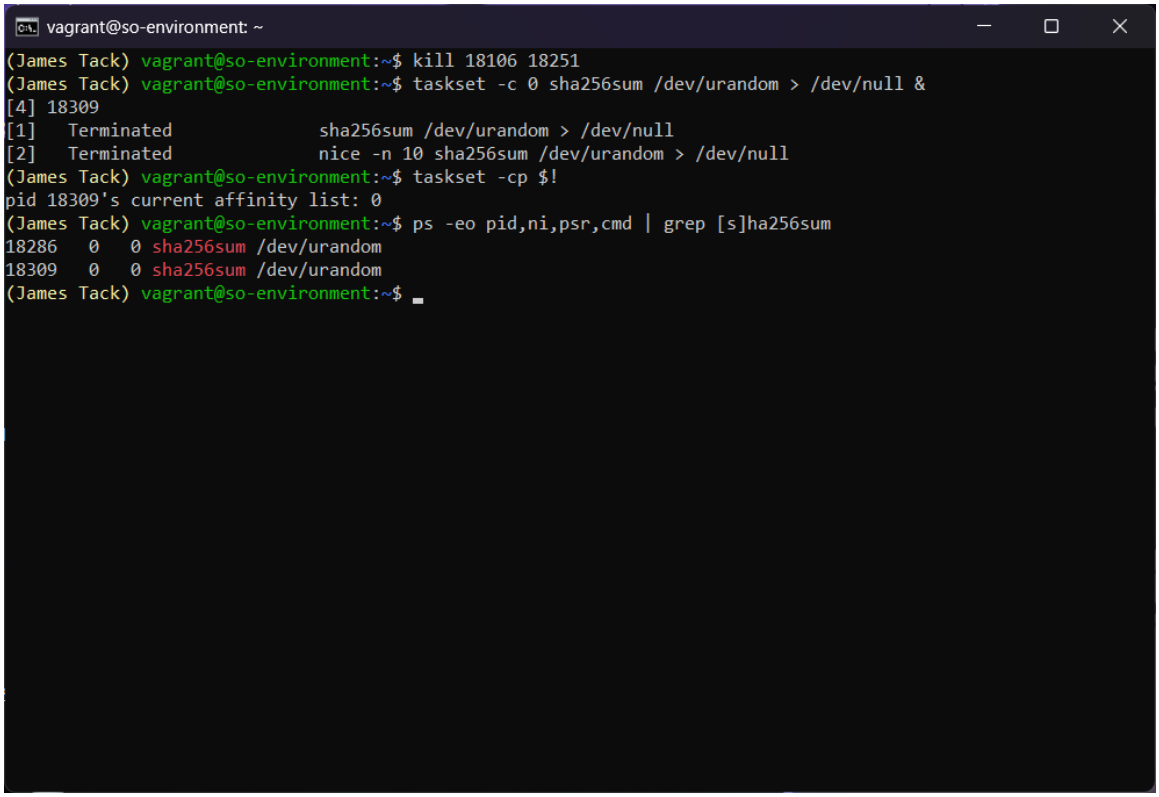



*Figura 8. Dos procesos pesados, los dos anclados al núcleo 0.*

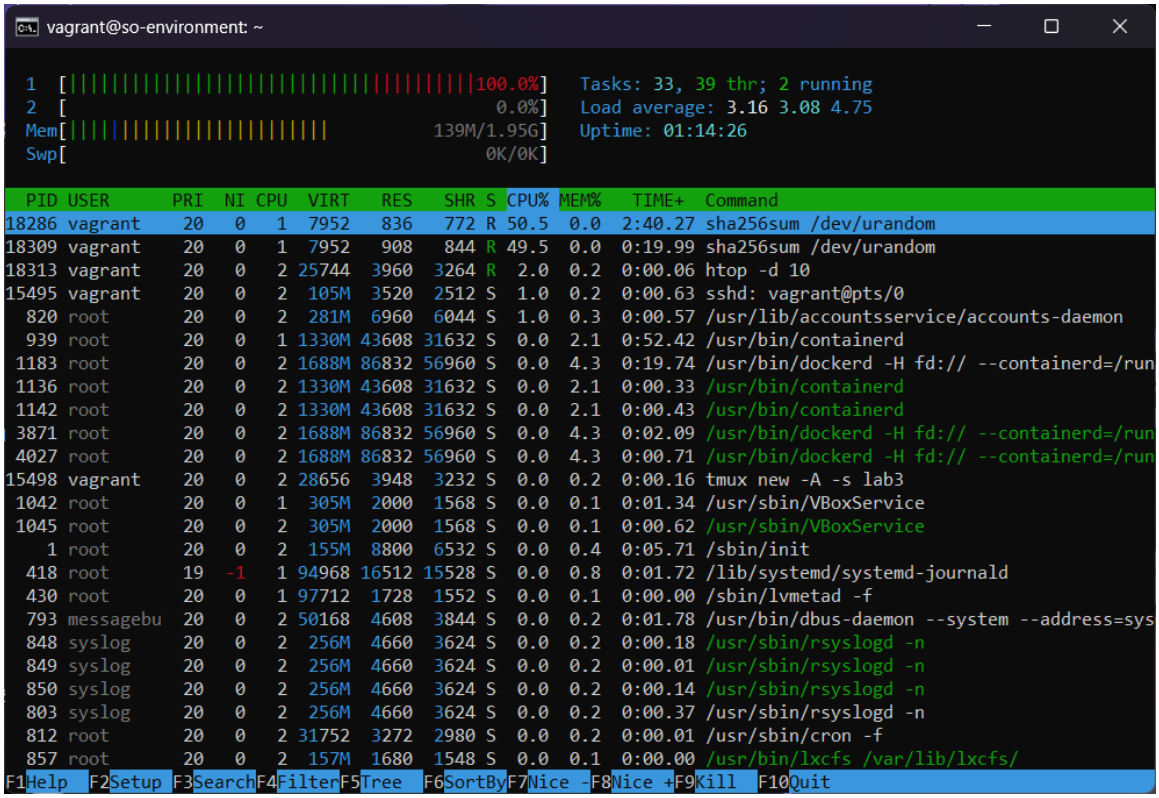



*Figura 9. htop: el núcleo 0 (CPU 1 en htop) al 100 % y el núcleo 1 sin carga.*

**Entregable 5. Análisis de taskset y afinidad**

1. `htop` numera los núcleos desde 1, así que su CPU 1 es el núcleo 0 de `taskset`. Ese núcleo está al 100 % y el otro al 0,0 %. Los dos `sha256sum` aparecen en la columna CPU con el valor 1 y se reparten el núcleo casi exacto a la mitad, 50,5 % y 49,5 %, que es lo que corresponde a dos tareas con el mismo nice y el mismo peso en CFS. En la vista anterior, con tres procesos sin restricción, los dos núcleos estaban al 100 %.

2. Anclar sirve cuando conviene que un proceso no salte de núcleo, por ejemplo para conservar la caché caliente de un servicio con mucho acceso a memoria, para reservar núcleos a una base de datos o a un proceso sensible a la latencia (como pide el gerente del caso), para encerrar un trabajo ruidoso en un núcleo y proteger al resto, para respetar la topología NUMA en servidores con varios sockets y para medir rendimiento sin la variación que introducen las migraciones.

3. Con dos núcleos, anclar un proceso pesado al núcleo 0 protege al resto del sistema, porque el núcleo 1 queda libre y ahí siguen corriendo la shell, SSH y los demonios. El costo es que el proceso anclado no puede usar el núcleo 1 aunque esté ocioso. La captura lo muestra: los dos procesos anclados tienen que conformarse con la mitad de un núcleo cada uno mientras el otro núcleo está al 0 %. Por lo tanto, el efecto sobre el rendimiento general depende de qué se busque. Si el objetivo es acotar el daño de un proceso problemático, como el de NovaTech, la afinidad es una buena herramienta. Si el objetivo es que ese trabajo termine rápido, se le está quitando la mitad de la máquina.

## Fase 5. Limpieza

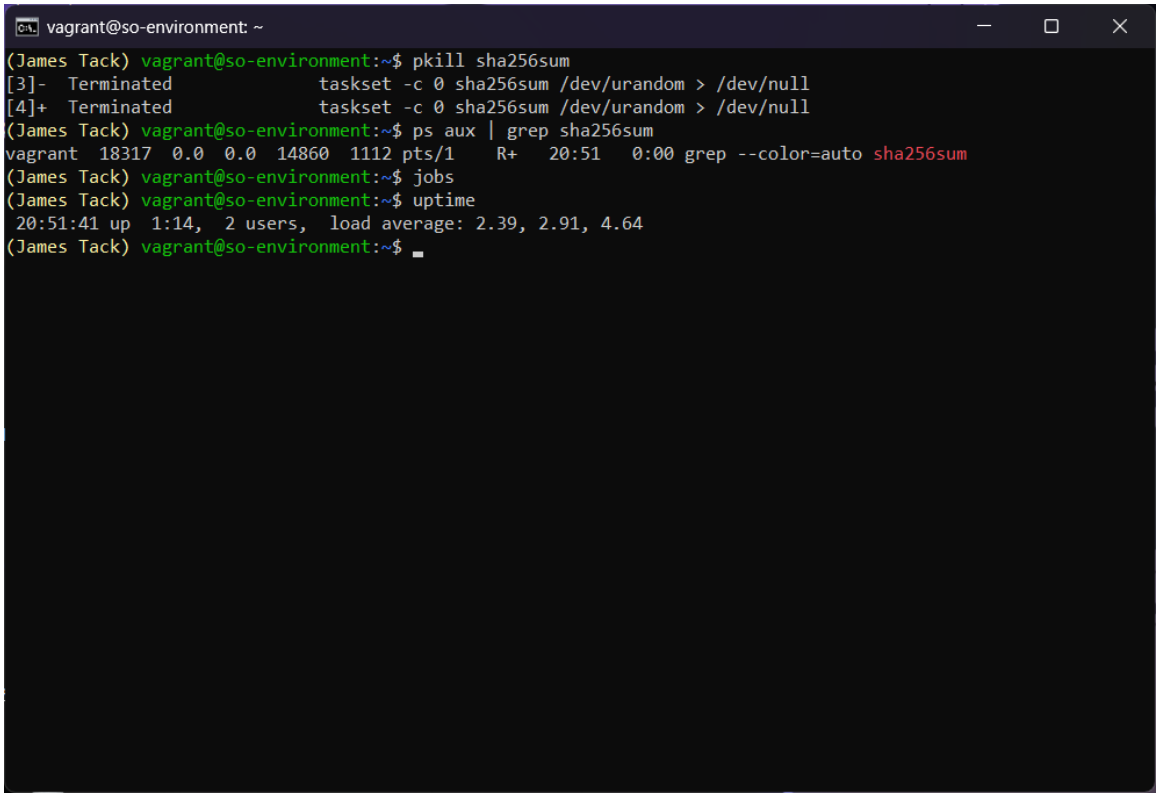



*Figura 10. Limpieza: solo queda el grep y la carga empieza a bajar.*

`ps aux` solo muestra el propio `grep`, `jobs` no lista trabajos pendientes y el promedio de carga de 1 minuto ya había bajado a 2,39. Solo terminé procesos lanzados por mi usuario, como pide la regla del laboratorio.

## Autocomprobación

| Paso | Lo que debía ver | Lo que vi | Figura |
|---|---|---|---|
| 1.2 ps -eo pid,ni,pri | NI 0, PRI cerca de 20 | NI 0, pri 19 (priority 20) | 2 y 3 |
| 2.2 nice -n 10 | NI 10 y PRI mayor que el normal | NI 10, priority 30 contra 20 (pri 9 contra 19) | 3 |
| 2.3 renice -n -5 | Permission denied | Permission denied | 4 |
| 3.1 chrt -p | SCHED_OTHER, prioridad 0 | SCHED_OTHER, prioridad 0 | 5 |
| 3.2 chrt -f | Error de permisos | Operation not permitted | 5 |
| 4.2 taskset -p | Solo el núcleo anclado | Máscara 1, lista 0 | 6 |

Los dos valores de `PRI` no coinciden con la tabla de la guía por la escala invertida de la columna `pri` de `ps`, explicada en la fase 1. La columna `priority` y `htop` sí dan lo que anticipa la guía.

## Cuestionario técnico

### 1. Niceness y prioridad

El proceso con nice -5 tiene un peso de 3121 en CFS y el de nice 15 tiene 36. Si compiten por el mismo núcleo, CFS les da tiempo en proporción a esos pesos, y el de nice -5 recibe alrededor de 3121/3157, un 98,9 % de la CPU, y el de nice 15 cerca del 1,1 %. El de nice 15 no se muere de hambre, porque CFS nunca deja un proceso listo sin turno, pero avanza muy despacio. Internamente cada tarea acumula un tiempo virtual (`vruntime`) que crece más rápido cuanto menor es su peso, y el planificador siempre elige la de menor `vruntime`, así que el de nice 15 alcanza su cuota en unos pocos milisegundos y vuelve a la fila. En cambio, si la máquina tiene dos núcleos libres, como la VM de este laboratorio, cada proceso toma uno y los dos corren al 100 %: el nice solo pesa cuando hay competencia.

### 2. Políticas de planificación

`SCHED_OTHER` alcanza para casi todo porque la mayoría de los procesos no tienen plazos estrictos. Un navegador, un compilador o un servidor web necesitan un reparto justo y una buena respuesta interactiva, y CFS ya da eso: favorece a las tareas que duermen mucho, impide que alguna se quede sin turno y deja ajustar el reparto con el nice sin riesgo para el sistema. `SCHED_FIFO` o `SCHED_RR` hacen falta cuando llegar tarde es un error y no solo una molestia. Es el caso del audio profesional, del control de un motor o de un brazo robótico y de la lectura de un sensor que tiene que muestrearse con un período fijo. FIFO sirve cuando la tarea hace un trabajo corto y cede la CPU sola; RR, cuando hay varias tareas de tiempo real con la misma prioridad que deben turnarse, ya que les impone un quantum entre ellas (100 ms por defecto en Linux).

### 3. Afinidad de CPU y locality of reference

Depende de cómo esté hecho el proceso. La localidad de referencia dice que un programa tiende a volver a los datos que usó hace poco y a los que están cerca en memoria. Si el proceso se queda siempre en el mismo núcleo, sus datos siguen en la L1 y la L2 de ese núcleo, y evita el costo de recargar la caché cada vez que el planificador lo mueve. Para un proceso de un solo hilo con mucho uso de caché, anclarlo tiene sentido.

Sin embargo, un proceso de machine learning casi nunca es de un solo hilo. Las bibliotecas de álgebra lineal reparten las operaciones entre todos los núcleos disponibles, y anclar ese proceso a un núcleo lo obliga a hacer en serie lo que podría hacer en paralelo en cuatro. Lo razonable es anclarlo a un grupo de núcleos que compartan la caché L3 o el mismo nodo NUMA, por ejemplo `taskset -c 1-3`, y dejar el núcleo 0 para el sistema. Además, el planificador de Linux ya intenta mantener cada tarea en el núcleo donde tiene la caché caliente y solo la migra cuando el desbalance de carga compensa el costo, así que anclar a mano conviene cuando hay otros procesos que la desplazarían.

### 4. Seguridad

Con `nice -n -20` un atacante se lleva la mayor parte de la CPU, pero CFS todavía reparte algo a los demás. Con `chrt -f 99` la situación es otra. Un proceso en loop infinito con prioridad de tiempo real máxima desplaza a todos los procesos normales de su núcleo. Si el atacante lanza una copia por

núcleo, todos los procesadores quedan ocupados por tareas de tiempo real y el sistema deja de atender SSH, la consola, el servidor web y buena parte de los hilos del kernel. Es una denegación de servicio por agotamiento de CPU, y el administrador ni siquiera puede entrar a matar los procesos. Combinado con un fork bomb sería peor todavía. Lo único que queda en pie es el 5 % que reserva `sched_rt_runtime_us`, y si el atacante también pudiera escribir en `/proc/sys`, ni eso. Por eso el kernel exige `CAP_SYS_NICE` para bajar el nice y para usar políticas de tiempo real, y por esa razón `RLIMIT_NICE` y `RLIMIT_RTPRIO` valen 0 para un usuario normal, como se vio en las fases 2 y 3.

## Conclusiones

La idea central de la práctica se vio con mis propios procesos: el usuario pide, el kernel decide. Pedí prioridad con `nice` y `renice`, pedí tiempo real con `chrt` y núcleos con `taskset`, y en todos los casos el kernel aplicó sus reglas: aceptó que cediera prioridad, me negó recuperarla aunque había sido mía, me negó el tiempo real y solo cambió de opinión cuando pasé por `sudo`.

Me llevo también que un mismo dato puede leerse al revés según la herramienta. La columna `pri` de `ps` y la columna `PRI` de `htop` muestran la misma prioridad en escalas opuestas, y la tabla de la guía solo cuadra si se mira la correcta. Antes de sacar una conclusión de un número hay que saber qué escala usa.

Lo que menos esperaba fue que el problema de rendimiento más serio del laboratorio no estuviera en Linux sino debajo. El planificador de la VM repartía bien, pero con las dos vCPU al 100 % la VM dejaba de recibir tiempo de procesador del anfitrión porque VirtualBox corre sobre Hyper-V. Es el mismo principio que en la pregunta de afinidad: quien reparte en un nivel depende de lo que le den en el nivel de abajo.

## Anexos

### Anexo A. Comandos ejecutados

| Fase | Comando | Para qué |
|---|---|---|
| Prep. | sudo docker stop tarea2-app | Detener la carga que dejó la Tarea 2 |
| Prep. | sudo apt install -y util-linux | Asegurar chrt y taskset |
| 1 | sha256sum /dev/urandom > /dev/null & | Generar carga de CPU |
| 1 | ps -eo pid,ni,pri,cmd | Ver niceness y prioridad |
| 2 | nice -n 10 sha256sum ... & | Lanzar un proceso con prioridad baja |
| 2 | ps -eo pid,ni,pri,priority,cls,cmd | Comparar las dos escalas de prioridad |
| 2 | renice -n -5 / 15 / 10 -p PID | Probar los límites de renice sin privilegios |
| 2 | ulimit -e | Ver el límite RLIMIT_NICE |
| 2 | sudo renice -n -5 -p PID | Subir la prioridad con privilegios |
| 3 | chrt -p PID / chrt -f -p 10 PID | Consultar y cambiar la política |
| 3 | chrt -m | Ver los rangos de prioridad por política |
| 3 | cat /proc/sys/kernel/sched_rt_runtime_us | Ver el límite de tiempo real |
| 3 | sudo chrt -f -p 10 PID (sobre sleep) | Confirmar que FIFO solo exige privilegios |

| Fase | Comando | Para qué |
|---|---|---|
| 4 | taskset -c 0 sha256sum ... & | Anclar un proceso al núcleo 0 |
| 4 | taskset -p PID / taskset -cp PID | Ver la afinidad como máscara y como lista |
| 4 | htop -d 10 | Ver el uso por núcleo |
| 5 | pkill sha256sum | Terminar los procesos de prueba |
| 5 | ps aux / jobs / uptime | Confirmar la limpieza |

## Anexo B. Salidas de texto

Las salidas clave de las capturas, en texto:

```
$ ps -eo pid,ni,pri,priority,cls,cmd | grep [s]ha256sum
18106   0  19  20  TS sha256sum /dev/urandom
18251  10   9  30  TS sha256sum /dev/urandom
$ renice -n 10 -p 18251
renice: failed to set priority for 18251 (process ID): Permission denied
$ ulimit -e
0
$ chrt -f -p 10 18106
chrt: failed to set pid 18106's policy: Operation not permitted
$ cat /proc/sys/kernel/sched_rt_runtime_us
950000
$ taskset -p 18286
pid 18286's current affinity mask: 1
```

Y la línea del log de VirtualBox (`VBox.log`) que explica el congelamiento con las dos vCPU saturadas:

```
HM: HMR3Init: Attempting fall back to NEM: AMD-V is not available
```